

SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# Python Programming
# Assignment -4
# CO-4


**Name: Hemnath R**

**Reg No:1920260116**


**Course Code:CSA0822**

**Github link:**https://github.com/hemnathr08-commits/python-programming-assignment_

**Topic:** File Processing, Exception Handling, Modular Programming and Robust Data Processing

I'll provide the report in the same format as Assignment 1: each question followed by its explanation, algorithm, complete Python code, sample input, output and conclusion wherever applicable.

# **TASK 1 – FILE-PROCESSING DESIGN REVIEW**

## **1.1 Diagnosis of the Given Script**

The original Python program has several problems that affect its performance, reliability and maintainability.

**Problems identified:**

1. **Hard-coded file path:** The program uses a fixed file location, making it difficult to execute on another computer.
2. **File not closed:** The file is opened without using a `with` statement, which may leave the file open.
3. **Memory consumption:** The `read()` method loads the entire file into memory.
4. **Bare exception handling:** Using `except: pass` hides errors and makes debugging difficult.
5. **Incorrect amount conversion:** Using `int()` does not support decimal amounts.
6. **Repeated string concatenation:** Repeatedly joining strings can reduce performance for large files.
7. **Blank-line handling:** Empty lines may cause incorrect processing.
8. **No error report:** Invalid records are ignored without recording the reason.

## **1.2 Corrected Python Program**

**Filename: **`task1.py`

```
from decimal import Decimal, InvalidOperationdef process_file(filename):
total = Decimal("0.00")    rejected = []    try:        with open(filename,
"r", encoding="utf-8") as file:            for line_no, line in
enumerate(file, start=1):                if not line.strip():
continue                try:                    parts =
line.strip().split(",")                    if len(parts) != 3:
raise ValueError("Expected 3 fields")                    register =
parts[0].strip()                    name = parts[1].strip()
amount = Decimal(parts[2].strip())                    if not register or not
name:                        raise ValueError("Empty field")
if not amount.is_finite():                            raise ValueError("Invalid
amount")                    total += amount
print(register, name, amount)                except (ValueError,
InvalidOperation) as error:
```

## **1.3 Sample Input**

**File: **`mess_oct.txt`

```
101,Arun,1250.50
102,Bala,980.00
103,Charan,1100.75
104,Divya,wrong
105,Ezhil
```

## **1.4 Expected Output**

```
101 Arun 1250.50
102 Bala 980.00
103 Charan 1100.75

Total Mess Bill: 3331.25

Rejected Records:
(4, 'Invalid amount')
(5, 'Expected 3 fields')
```

## **1.5 Complexity Analysis**

- **Time Complexity:** O(n), because every line is processed once.
- **Space Complexity:** O(n) for storing accepted and rejected records in memory.
- **Improvement:** Reading line by line reduces memory usage compared with loading the entire file.

## **1.6 Conclusion**

The corrected program improves reliability by using exception handling, Decimal conversion, file management and line-by-line processing. It can handle large files and report invalid records without stopping the complete execution.

# **TASK 2 – FROM REQUIREMENT TO FILE-PROCESSING DESIGN**

## **2.1 Problem Description**

A college uses daily biometric attendance files to maintain student attendance records. The objective is to combine multiple daily files into one monthly attendance report.

The system should:

- Read daily attendance files.
- Identify student register numbers.
- Count present and absent days.
- Calculate attendance percentage.
- Identify damaged records.
- Generate a monthly CSV report.

## **2.2 Data Structure Used**

A Python dictionary is used to store student attendance.

Example:

```
attendance = {    "192311001": {"P": 2, "A": 1},    "192311002": {"P": 2, "A": 1}}
```

Attendance percentage formula:

\\[ \text{Attendance Percentage} = \frac{\text{Present Days}}{\text{Total Days}}\times100 \\]

## **2.3 Algorithm**

1. Start the program.
2. Create an empty dictionary.
3. Read all daily attendance files.
4. Process each line.
5. Validate the number of fields.
6. Identify present and absent status.
7. Update the dictionary.
8. Record invalid lines in an error list.
9. Calculate attendance percentages.
10. Generate CSV reports.
11. Stop the program.

## **2.4 Python Program**

**Filename: **`task2.py`

```
import osimport csvfrom collections import defaultdictdef
process_attendance(folder):    attendance = defaultdict(lambda: {"P": 0, "A":
0})    errors = []    for filename in os.listdir(folder):        if not
filename.endswith(".txt"):            continue        path =
os.path.join(folder, filename)        try:            with open(path, "r",
encoding="utf-8") as file:          line_no,  for  line.strip():  in
enumerate(file, 1):                 line  if  not
continue  if  len(parts)    parts =
!=   3:   errors.append(
fields") register,                        (filename, line_no, "Invalid
parts ```                       )                          continue
          date, status = [                        p.strip() for p in
## **2.5 Sample Input**  ]
**day1.txt**

```  192311001,2026-10-
01,P  192311002,2026-10-
01,A  192311003,2026-10-
01,P BAD-LINE
```

**day2.txt**

```  192311001,2026-10-
02,P  192311002,2026-10-
02,P  192311003,2026-10-
02,A  192311004,2026-10-
02,P ```

**day3.txt**

```  192311001,2026-10-
03,A
```

```
192311002,2026-10-03,P
192311003,2026-10-03,P
192311004,2026-10-03,L
```

## **2.6 Expected Output**

```
Attendance Report Generated
Error Log Generated
```

**Generated CSV report:**

| Register Number | Present | Absent | Percentage |
| --- | --- | --- | --- |
| 192311001 | 2 | 1 | 66.67% |
| 192311002 | 2 | 1 | 66.67% |
| 192311003 | 2 | 1 | 66.67% |
| 192311004 | 1 | 0 | 100.00% |

The error log contains the damaged line in day1 and invalid status in day3.

## **2.7 Complexity Analysis**

- Time Complexity: O(n), where n is the total number of attendance records.
- Space Complexity: O(s + e), where s represents students and e represents invalid records.
- A dictionary provides efficient student-wise counting without repeatedly scanning all files.

## **2.8 Conclusion**

The attendance processing system combines daily files, calculates attendance percentages and identifies damaged records. CSV reports make the results easy to view and analyse.

# **TASK 3 – DESIGN THE ERROR HANDLING**

## **3.1 Problem Description**

A college attendance system processes student attendance records stored in text files. Sometimes the files may contain incorrect data, missing fields, invalid attendance status or file access errors.

The objective is to develop a Python program that identifies errors and handles them without stopping the complete execution.

## **3.2 Types of Errors Identified**

| Error | Description |
| --- | --- |
| FileNotFoundError | Inputfile does not exist |
| PermissionError | File cannot be accessed |
| UnicodeDecodeError | File contains invalid text encoding |
| ValueError | Incorrect data value |
| IndexError | Accessing a missing list element |
| InvalidRecord | Custom exception for damaged attendance records |

## **3.3 Custom Exception**

A custom exception called `InvalidRecord` is created by inheriting from Python's built-in `Exception` class.

It is raised when:

- The record does not contain three fields.
- Register number is empty.
- Date is empty.
- Attendance status is neither P nor A.

## **3.4 Python Program**

**Filename: **`task3.py`

```
class InvalidRecord(Exception):    passdef parse_line(line):    parts = line.strip().split(",")    if len(parts) != 3:        raise InvalidRecord(            f"expected 3 fields, found {len(parts)}"        )register, date, status = [        p.strip() for p in parts    ]    if not register or not date or status not in ("P", "A"):        raise InvalidRecord("invalid status or empty field")    return register, date, statusdef summarise(path):    present = 0    absent = 0    errors = []    try:        with open(path, "r", encoding="utf-8") as file:            for line_no, line in enumerate(file, 1):                if not line.strip():                    continue
```

## **3.5 Sample Input**

```
192311001,2026-10-08,P
192311002,2026-10-08,A
192311003,2026-10-08
192311004,2026-10-08,L
```

## **3.6 Expected Output**

```
(1, 1, [(3, 'expected 3 fields, found 2'),
(4, 'invalid status or empty field')])

(0, 0, [(0, 'File not found')])
```

## **3.7 Explanation of Error Handling**

**try:** Contains the code that may generate an exception.

**except:** Handles the exception when an error occurs.

**raise:** Creates and raises a custom exception.

**finally:** Executes cleanup operations whether an exception occurs or not. In this program, `with open()` automatically handles file closing.

## **3.8 Experiment – Misuse of Exception Handling**

If the program uses:

```
try:    process_file()except:    pass
```

all errors are silently ignored. The programmer cannot identify which record failed or why it failed.

Instead, specific exception handling provides meaningful error messages and makes the program easier to debug.

## **3.9 Complexity Analysis**

- Time Complexity: O(n), because every record is processed once.
- Space Complexity: O(e), where e represents the number of rejected records.
- Custom exceptions improve error identification and program reliability.

## **3.10 Conclusion**

The program demonstrates the use of built-in and user-defined exceptions in Python. It identifies invalid attendance records, handles missing files and continues processing valid records without terminating the entire program.

# **TASK 4 – FILE AND EXCEPTION FORENSICS**

## **4.1 Problem Description**

A college library maintains daily book-return records. The existing program contains file-handling errors that may cause previous records to be lost or book names to be joined together.

The objective is to identify these defects and develop a corrected program that safely maintains all daily return records.

## **4.2 Identification of Problems**

1. **Incorrect file mode:** Using write mode (`w`) deletes existing records.
2. **Missing newline:** Book names may be joined together.
3. **Improper reading:** Using `split()` can separate book titles containing spaces.
4. **Improper file handling:** Files should be opened using a `with` statement.
5. **Insufficient exception handling:** Missing files and permission errors should be handled.
6. **Loss of previous records:** Daily records must be preserved in the log.

## **4.3 Corrected Python Program**

**Filename: **`task4.py`

```
def save_returns(books, log_file):    with open(log_file, "a", encoding="utf-8") as log:        for book in books:            log.write(book.strip() + "\n")def process(day_file, log_file):    try:        with open(day_file, "r", encoding="utf-8") as file:            books = [                line.strip() for line in file                if line.strip()            ]        save_returns(books, log_file)        print("Processed:", day_file)    except FileNotFoundError:        print("Missing file:", day_file)    except PermissionError:        print("Permission denied:", day_file)    except OSError as error:        print("File error:", error)if __name__ == "__main__":    days = ["day1.txt", "day2.txt", "day3.txt"]
```

## **4.4 Sample Input**
**day1.txt**

```
The Alchemist
Wings of Fire
```

**day2.txt**

```
The Guide
```

**day3.txt**

```
Ponniyin Selvan
```

## **4.5 Expected Output**

```
Processed:         day1.txt
Processed:         day2.txt
Processed: day3.txt

Final Return Log:

The Alchemist
Wings of Fire
The Guide
Ponniyin Selvan
```

## **4.6 File Modes Used**

| File Mode | Purpose |
| --- | --- |
| `r` | Read an existing file |
| `w` | Write and replace existing contents |
| `a` | Append new data without deleting existing contents |
| `r+` | Read and write an existing file |

In this program, append mode (`a`) is used to preserve earlier book-return records.

## **4.7 Alternative Method**

The `try-finally` method can also be used to close files manually.

Example:

```
     = Nonetry:    file = open("day1.txt", "r")    print(file.read())finally:
if file:        file.close()
file
```

However, the `with` statement is preferred because it automatically closes the file.

## **4.8 Complexity Analysis**

- Time Complexity: O(n), where n represents the total number of book records.
- Space Complexity: O(n) for storing the daily book records in the list.
- Append mode prevents the loss of previously stored records.

## **4.9 Conclusion**

The corrected library system safely processes daily book-return records and maintains a continuous log. Proper file modes, exception handling and newline characters improve the reliability of the application.

# **TASK 5 – THE SAME UTILITY ORGANISED FOUR WAYS**

## **5.1 Introduction**

Python supports different ways of organising programs. A program can be written in a single script, divided into multiple modules or organised as a package.

This task demonstrates four different structures using a student attendance management utility.

The utility performs three major operations:

1. Reading attendance records.
2. Calculating attendance percentages.
3. Generating attendance reports.

## **5.2 Organisation A – Single Script**

In this method, all functions are written in one Python file.
**Advantages:**

- Easy to create and execute.
- No module imports are required.
- Suitable for small programs.

**Disadvantages:**

- Difficult to maintain large programs.
- Code reuse is limited.
- Testing individual functions becomes difficult.

### **Python Program**

**Filename: **`task5_single.py`

```
from collections import defaultdictimport osdef read_files(folder):
= defaultdict(lambda: {"P": 0, "A": 0})                                    counts
os.listdir(folder):                          for filename in
open(os.path.join(folder,  if filename.endswith(".txt"):           with
file:                      filename), "r") as file:               for line in
                         parts = line.strip().split(",")
if len(parts) == 3 and parts[2] in ("P", "A"):
counts[parts[0]][parts[2]] += 1    return countsdef
calculate_percentage(counts):    result = {}    for reg, data in
counts.items():        total = data["P"] + data["A"]        result[reg] =
(            data["P"] / total * 100            if total else 0        )
```

### **Expected Output**

```
Report Generated Successfully
```

## **5.3 Organisation B – Main Script with Imported Modules**

In this method, the program is divided into separate Python files. The main script imports the required modules and executes their functions.

**Advantages:**

- Improves code organisation.
- Functions can be reused.
- Easier debugging and testing.

**Disadvantages:**

- Requires multiple files.
- Incorrect module names can cause import errors.

### **File 1 – reader.py**

```
def read_data():    data = []    with open("attendance.txt",  "r") as file:
for line in file:             data.append(line.strip().split(","))    return
data
```

### **File 2 – stats.py**

```
def calculate(data):    result = {}    for row in data:        if len(row) ==
3 and row[2] in  ("P", "A"):            reg = row[0]            if reg not in
result:                result[reg] = {"P": 0, "A": 0}
result[reg][row[2]] += 1    return result
```

### **File 3 – main.py**

```
import readerimport statsdata = reader.read_data()result =
stats.calculate(data)for reg, value in result.items():    total = value["P"]
+ value["A"]                = value["P"] / total * 100    print(reg,
percentage,
round(percentage, 2), "%")
```

### **Sample Input – attendance.txt**

```
192311001,2026-10-01,P
192311001,2026-10-02,P
192311001,2026-10-03,A
192311002,2026-10-01,A
192311002,2026-10-02,P
```

### **Expected Output**

```
192311001 66.67 %
192311002 50.0 %
```

## **5.4 Organisation C – Three Modules in One Folder**

In this method, the application is divided into three independent modules and one main program.

### **Folder Structure**

```
Attendance/
main.py
reader.py
stats.py
```

```
    report.py
    attendance.txt
```

### **Module 1 – reader.py**
```
def
[   read_data(filename):    with open(filename, "r") as file:        return
```          line.strip().split(",")            for line in file        ]

### **Module 2 – stats.py**

```
def
    calculate(data):    result = {}for     row in data:        if len(row) ==
3 and row[2] in  ("P", "A"):        reg     = row[0]            if reg not in
result:                 result[reg] ₹"P":   0, "A": 0}
result[reg][row[2]] += 1    return resultdef percentage(result):    output =
{}    for reg, value in result.items():        total = value["P"] +
value["A"]        output[reg] = value["P"] / total * 100    return output
```

### **Module 3 – report.py**

```
def write_report(data):    with open("attendance_report.txt", "w") as file:
for reg, pct in data.items():            file.write(f"{reg}: {pct:.2f}%\n")
```

### **Main Program – main.py**

```
import          readerimport          statsimport          reportdata          =
reader.read_data("attendance.txt")result  =  stats.calculate(data)percentages  =
stats.percentage(result)report.write_report(percentages)print("Attendance
Report Created")
```

### **Expected Output**

```
Attendance Report Created
```

**Generated file: attendance_report.txt**

```
192311001: 66.67%
192311002: 50.00%
```

## **5.5 Organisation D – Python Package**

A Python package is a collection of related modules organised inside a directory.

It normally contains an `__init__.py` file that identifies the directory as a regular Python package.

### **Folder Structure**

```
AttendancePackage/
    main.py
```

```
    attendance_pkg/
    __init__.py
    reader.py
    stats.py
    report.py
    attendance.txt
```

### **Module 1 – attendance_pkg/init.py**
```
# Attendance Processing Package
```

### **Module 2 – attendance_pkg/reader.py**
```
def
[
``` read_data(filename):    with open(filename, "r") as file:        return
            line.strip().split(",")            for line in file        ]
### **Module 3 – attendance_pkg/stats.py**

```
def

    calculate(data):    result = {}for     row in data:        if len(row) ==
3 and row[2] in  ("P", "A"):        reg     = row[0]            if reg not in
result:                result[reg] ∉"P":   0, "A": 0}
result[reg][row[2]] += 1    return resultdef percentage(result):    output =
{}    for reg, value in result.items():        total = value["P"] +
value["A"]        output[reg] = value["P"] / total * 100    return output
```

### **Module 4 – attendance_pkg/report.py**

```
def write_report(data):    with open("attendance_report.txt", "w") as file:
for reg, pct in data.items():            file.write(f"{reg}: {pct:.2f}%\n")
```

### **Main Program – main.py**

```
from attendance_pkg.reader import read_datafrom attendance_pkg.stats import
calculate, percentagefrom attendance_pkg.report import write_reportdata =
read_data("attendance.txt")result       =       calculate(data)output       =
percentage(result)write_report(output)print("Package   Execution   Successful")
```

### **Expected Output**

```
Package Execution Successful
```

## **5.6 Change the Requirement – Three Consecutive Absences**

The college introduces a new requirement: the attendance system must identify students who are absent for three consecutive days.

To implement this feature, the program needs to maintain each student's daily attendance history.

### **Python Program – Consecutive Absence Detection**

```
def find_consecutive_absences(records):    history = {}    for reg, date,
status in records:        if reg not in history:            history[reg] = []
history[reg].append(status)    result = []    for reg, statuses in
history.items():        count = 0        for status in statuses:
if status == "A":                count += 1            else:
count = 0            if count == 3:                result.append(reg)
break    return resultrecords = [    ("192311001", "Day1", "A"),
("192311001", "Day2", "A"),    ("192311001", "Day3", "A"),    ("192311002",
"Day1", "P"),
```

### **Expected Output**

```
Students with three consecutive absences:
['192311001']
```

The example assumes the records are supplied in chronological order.
## **5.7 Comparison of Four Organisations**

| Feature | Single Script | Imported Modules | Three Modules | Package |
| --- | --- | --- | --- | --- |
| Structure | One file | Multiple files | Separate modules | Package directory |
| Reusability | Low | High | High | High |
| Maintenance | Difficultfor large programs | Easy | Easy | Very easy |
| Scalability | Limited | Good | Good | Excellent |
| Complexity | Low | Medium | Medium | Medium |

## **5.8 Memory and Performance Comparison**

- A single script has less module-loading overhead.
- Multiple modules introduce a small import overhead.
- Packages provide a structured way to organise large applications.
- All four approaches can use the same attendance calculation algorithm.
- The main difference is code organisation, maintainability and reusability.

## **5.9 Conclusion**

This task demonstrates four different ways of organising Python programs. Single-script programming is suitable for small applications, while modular programming and packages are more suitable for larger applications. The package structure provides better organisation, scalability and code reuse.

# **TASK 6 – DESIGN CHALLENGE: COLLEGE FEST REGISTRATION DESK**

13 Marks

## **6.1 Stage 1 – Problem Understanding**

A college organises an annual technical fest in which students from different departments participate in various events.

The registration details are stored in separate text files for five departments:

1. CSE – Computer Science and Engineering
2. ECE – Electronics and Communication Engineering
3. BIO – Bioinformatics
4. MECH – Mechanical Engineering
5. CIVIL – Civil Engineering

The objective is to develop a Python-based registration management system that collects all department records, validates them and generates organised reports.

**Input format:**

```
Register Number;Name;Department;Email;Events
```

**Example:**

```
F001;Anu;CSE;anu@example.com;Coding|Quiz
```

## **6.2 Stage 2 – Data Processing Flow**

The system follows these steps:

1. Read all five department files.
2. Extract student registration details.
3. Validate the number of fields.
4. Check the e-mail address.
5. Identify duplicate registration numbers.
6. Accept valid records.
7. Reject invalid records.
8. Generate a master registration CSV file.
9. Generate event-wise participant lists.
10. Generate a summary report.
11. Generate an error log.

### **Algorithm**

```
START

Read department files

FOR each department file:

    Read student records

    FOR each record:

        Validate record format

        IF record is damaged:
            Add to error log
            Continue

        Validate email

        IF email is invalid:
        Add to error log
        Continue
```

```
        Check duplicate registration number

        IF duplicate:
            Add to error log
            Continue

        Store valid registration

Generate master CSV

Generate event-wise participant files

Generate summary report

Generate error log

STOP
```

## **6.3 Stage 3 – Critical Design Decisions**

The following Python concepts are used in the system.

| Python Concept | Purpose |
| --- | --- |
| File Handling | Read department files |
| Functions | Divide the application into modules |
| Lists | Store event information |
| Dictionaries | Store student details |
| Sets | Identify duplicate registrations |
| Exception Handling | Handle missing files and damaged records |
| Regular Expressions | Validate e-mail addresses |
| CSV Module | Generate master registration reports |
| os Module | Manage folders and file paths |

**Duplicate identification:**

A Python set is used to store registration numbers.

```
seen = set()if register in seen:    print("Duplicate Registration")else:
seen.add(register)
```

This avoids repeatedly searching through all previously accepted registrations.

## **6.4 Stage 4 – Package Design**

The application is divided into four Python modules.

### **Folder Structure**

```
task6_fest/
│
├── main.py
├── reader.py
├── validator.py
├── writer.py
│
├── input/
│   ├── CSE.txt
│   ├── ECE.txt
```

```
│   ├── BIO.txt
│   ├── MECH.txt
│   └── CIVIL.txt
│
└── output/
```

**Module Description:**

| Module | Responsibility |
| --- | --- |
| main.py | Controls complete execution |
| reader.py | Reads department files |
| validator.py | Validates student information |
| writer.py | Generates CSV and text reports |

### **Module 1 – reader.py**

```
import osdef read_department_files(folder):    departments = ["CSE", "ECE", "BIO", "MECH", "CIVIL"]    for dept in departments:        filename = os.path.join(folder, dept + ".txt")        try:            with open(filename, "r", encoding="utf-8") as file:                for line_no, line in enumerate(file, 1):                    if not line.strip():                        continue                    fields = [                        x.strip() for x in line.strip().split(";")                    ]                    if len(fields) != 5:                        yield filename, line_no, None                    else:                        yield filename, line_no, fields        except (FileNotFoundError, PermissionError,                UnicodeDecodeError, OSError):            yield filename, 0, None
```

### **Module 2 – validator.py**

```
import redef validate_record(fields):    if len(fields) != 5:        raise ValueError("Invalid number of fields")    register, name, department, email, events = fields    if not all([register, name, department, email, events]):        raise ValueError("Empty field")    pattern = (        r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"    )    if not re.fullmatch(pattern, email):        raise ValueError("Invalid email")    event_list = [e.strip() for e in events.split("|")]    if any(not event for event in event_list):        raise ValueError("Invalid event")    return {        "register": register,        "name": name,        "department": department,        "email": email,        "events": event_list    }
```

### **Module 3 – writer.py**

```
import osimport csvimport redef write_outputs(records, errors, folder):    os.makedirs(folder, exist_ok=True)    # Generate Master CSV    with open(        os.path.join(folder, "master_list.csv"),        "w",        newline="",        encoding="utf-8"    ) as file:        writer = csv.writer(file)        writer.writerow([            "Register",            "Name",            "Department",            "Email",            "Events"        ])        for r in records:            writer.writerow([                r["register"],                r["name"],                r["department"],                r["email"],                "|".join(r["events"])            ])
```

### **Module 4 – main.py**

```
from reader import read_department_filesfrom validator import
validate_recordfrom writer import write_outputsdef run(input_folder,
output_folder):    records = []    errors = []    seen = set()    for
filename, line_no, fields in read_department_files(input_folder):        if
fields is None:            reason = (                "Missing or unreadable
department file"                if line_no == 0                else "Damaged
line"            )            errors.append((filename, line_no, reason))
continue        try:            record = validate_record(fields)
register = record["register"]            if register in seen:
errors.append((                    filename,                    line_no,
"Duplicate registration"
```

## **6.5 Stage 5 – Implementation and Demonstration**

### **Sample Input Files**

**CSE.txt**

```
F001;Anu;CSE;anu@example.com;Coding|Quiz
F002;Bala;CSE;bala@example.com;Quiz|Debugging
F001;Anu;CSE;anu@example.com;Coding
F003;Charan;CSE;charan-at-mail;Coding
DAMAGED;LINE
```

**ECE.txt**

```
F004;Divya;ECE;divya@example.com;Circuit Design|Quiz
```

**BIO.txt**

```
F005;Ezhil;BIO;ezhil@example.com;Bio Innovation
```

**MECH.txt**

```
F006;Farhan;MECH;farhan@example.com;Robotics
```

For testing, the CIVIL file is intentionally left missing.
### **Expected Output**

```
Fest Registration Completed
Valid Registrations: 5
Rejected Records: 4
Reports Generated Successfully
```

### **Generated Output Files**

**master_list.csv**

Contains all valid student registration details.
**Event-wise text files**

Contains participant details for each selected event.

**summary.txt**

Contains the total valid registrations, rejected records and events.
**error_log.csv**

Contains duplicate registrations, invalid e-mails, damaged records and missing files.

### **Expected Summary Report**

```
Valid registrations: 5
Rejected records: 4
Total events: 6
```

Error Conditions Tested

| Test Case | Expected Result |
| --- | --- |
| Valid registration | Accepted |
| Duplicate register number | Rejected |
| Invalid e-mail | Rejected |
| Damaged record | Rejected |
| Missing department file | Logged |
| Valid multiple events | Added to respective event lists |

6.6 Complexity Analysis
Let n represent the total number of student registrations.

- **Time Complexity:** O(n + e), on average, where e is the total number of event selections.
- **Space Complexity:** O(n + e), for storing accepted registrations and event information.
- A set is used for efficient duplicate detection.
- Modular programming makes the application easier to maintain and extend.

6.7 Conclusion

The College Fest Registration Desk system demonstrates how Python can be used to develop a practical registration management application. It processes multiple department files, validates student details, identifies duplicate registrations and generates master and event-wise reports. The modular structure improves code readability, reusability and maintainability.